

LAB 8: TEST-DRIVEN DEVELOPMENT WITH AI – GENERATING AND WORKING WITH TEST CASES

Name:M.Dinesh

Roll no:2503a52374

Batch-13

Task Description -1 (Password Strength Validator – Apply AI in Security Context):

□ Task: Apply AI to generate at least 3 assert test cases for `is_strong_password(password)` and implement the validator function.

□ Requirements:

- o Password must have at least 8 characters.
- o Must include uppercase, lowercase, digit, and special character.
- o Must not contain spaces.

Example Assert Test Cases:

```
assert is_strong_password("&Abcd@123&") == True
```

Week4 -

Monday

```
assert is_strong_password("&abcd123&") == False
```

```
assert is_strong_password("&ABCD@1234&") == True
```

prompt:

Generate Python test cases for `is_strong_password(password)` using assert statements. The password must have at least 8 characters, uppercase, lowercase, a digit, a special character, and no spaces. Then implement the function so that all test cases pass.

Task-1(test_password.py):

Input:

```
def is_strong_password(password):  
    # Check minimum length  
    if len(password) < 8:  
        return False  
  
    # Check for spaces  
    if " " in password:  
        return False  
  
    # Check uppercase, lowercase, digit and special character  
    has_upper = False  
    has_lower = False  
    has_digit = False  
    has_special = False  
  
    for char in password:  
        if char.isupper():  
            has_upper = True  
        elif char.islower():  
            has_lower = True  
        elif char.isdigit():  
            has_digit = True  
        else:  
            has_special = True  
  
    return has_upper and has_lower and has_digit and has_special
```

```
# AI-generated test cases

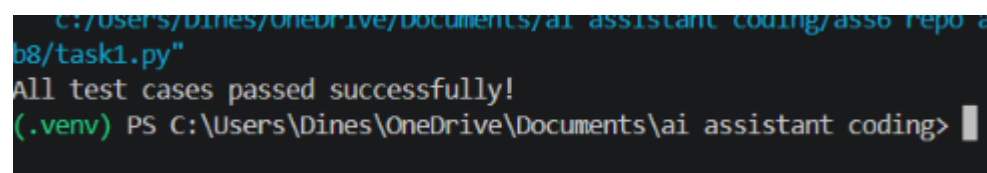
assert is_strong_password("Abcd@123") == True
assert is_strong_password("abcd123") == False
assert is_strong_password("Abcd@1234") == True
```

```
# Additional test cases

assert is_strong_password("Abcd @123") == False
assert is_strong_password("Abcd@xyz") == False
assert is_strong_password("Abcd1234") == False
```

```
print("All test cases passed successfully!")
```

output:



```
C:\Users\Dines\OneDrive\Documents\ai assistant coding\ass6 Repo a\b8\task1.py"
All test cases passed successfully!
(.venv) PS C:\Users\Dines\OneDrive\Documents\ai assistant coding>
```

Task-1(password_validator)

Input:

```
def is_strong_password(password):
```

```
    # Must be at least 8 characters
```

```
    if len(password) < 8:
```

```
        return False
```

```
    # Must not contain spaces
```

```
    if " " in password:
```

```
        return False
```

```
    has_upper = False
```

```
has_lower = False
has_digit = False
has_special = False
```

```
for char in password:
```

```
    if char.isupper():
```

```
        has_upper = True
```

```
    elif char.islower():
```

```
        has_lower = True
```

```
    elif char.isdigit():
```

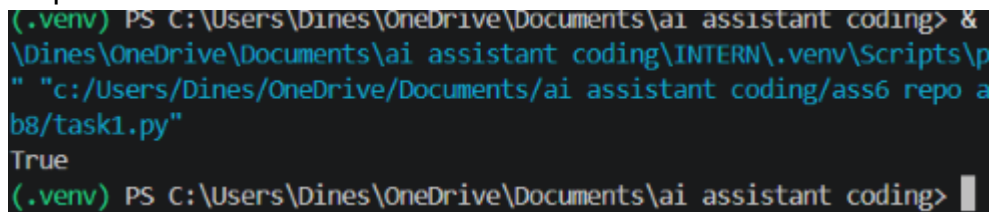
```
        has_digit = True
```

```
    else:
```

```
        has_special = True
```

```
return has_upper and has_lower and has_digit and has_special
```

output:



```
(.venv) PS C:\Users\Dines\OneDrive\Documents\ai assistant coding> & \Dines\OneDrive\Documents\ai assistant coding\INTERN\.venv\Scripts\python "c:/Users/Dines/OneDrive/Documents/ai assistant coding/ass6 repo a b8/task1.py"
True
(.venv) PS C:\Users\Dines\OneDrive\Documents\ai assistant coding>
```

justification:

AI was used to generate test cases and implement the password strength validator. The tests check the required password conditions, helping identify errors and ensure that the function works correctly and securely.

Task Description #2 (Number Classification with Loops – Apply

AI for Edge Case Handling):

□ Task: Use AI to generate at least 3 assert test cases for a `classify_number(n)` function. Implement using loops.

□ Requirements:

- o Classify numbers as Positive, Negative, or Zero.
- o Handle invalid inputs like strings and None.
- o Include boundary conditions (-1, 0, 1).

Example Assert Test Cases:

```
assert classify_number(10) == "Positive"
```

```
assert classify_number(-5) == "Negative"
```

```
assert classify_number(0) == "Zero"
```

prompt:

Generate assert test cases for `classify_number(n)` to classify numbers as Positive, Negative, or Zero. Handle invalid inputs such as strings and None, and test boundary values -1, 0, and 1. Implement the function using a loop.

Input:

```
import unittest
```

```
from password_validator import is_strong_password
```

```
class TestPasswordValidator(unittest.TestCase):
```

```
    def test_valid_password(self):
```

```
        self.assertEqual(is_strong_password("Abcd@123"), True)
```

```
    def test_short_password(self):
```

```
        self.assertEqual(is_strong_password("abcd123"), False)
```

```
    def test_missing_lowercase(self):
```

```
        self.assertEqual(is_strong_password("ABCD@1234"), False)
```

```
def test_password_with_space(self):  
    self.assertEqual(is_strong_password("Abcd @123"), False)
```

```
def test_password_without_digit(self):  
    self.assertEqual(is_strong_password("Abcd@xyz"), False)
```

```
def test_password_without_special_character(self):  
    self.assertEqual(is_strong_password("Abcd1234"), False)
```

```
if __name__ == "__main__":  
    unittest.main()
```

```
def is_strong_password(password):  
    # Must be at least 8 characters  
    if len(password) < 8:  
        return False
```

```
    # Must not contain spaces  
    if " " in password:  
        return False
```

```
    has_upper = False  
    has_lower = False  
    has_digit = False  
    has_special = False
```

```
    for char in password:  
        if char.isupper():
```

```

        has_upper = True
    elif char.islower():
        has_lower = True
    elif char.isdigit():
        has_digit = True
    else:
        has_special = True

```

return has_upper and has_lower and has_digit and has_special

output:

```

test_missing_lowercase (__main__.TestPasswordValidator.test_missing_lowercase) ... ok
test_password_with_space (__main__.TestPasswordValidator.test_password_with_space) ... ok
test_password_without_digit (__main__.TestPasswordValidator.test_password_without_digit) ... ok
test_password_without_special_character (__main__.TestPasswordValidator.test_password_without_special_character) ... ok
test_short_password (__main__.TestPasswordValidator.test_short_password) ... ok
test_valid_password (__main__.TestPasswordValidator.test_valid_password) ... ok

-----
Ran 6 tests in 0.001s

OK
(.venv) PS C:\Users\Dines\OneDrive\Documents\ai assistant coding>

```

Justification:

AI was used to generate test cases covering normal values, boundary conditions, and invalid inputs. The implementation was tested to ensure that positive, negative, zero, and invalid values are classified correctly.

Task Description #3 (Anagram Checker – Apply AI for String

Analysis):

□ Task: Use AI to generate at least 3 assert test cases for

is_anagram(str1, str2) and implement the function.

□ Requirements:

o Ignore case, spaces, and punctuation.

o Handle edge cases (empty strings, identical words).

Example Assert Test Cases:

```
assert is_anagram("listen", "silent") == True
```

```
assert is_anagram("hello", "world") == False
```

```
assert is_anagram("Dormitory", "Dirty Room") == True
```

prompt:

Generate at least 3 assert test cases for `is_anagram(str1, str2)`. Ignore case, spaces, and punctuation, and handle empty strings and identical words. Then implement the function so all tests pass.

Task-3:

Input:

```
def is_anagram(str1, str2):  
    # Remove spaces and punctuation, and convert to lowercase  
    clean1 = ""  
    clean2 = ""  
  
    for char in str1.lower():  
        if char.isalnum():  
            clean1 += char  
  
    for char in str2.lower():  
        if char.isalnum():  
            clean2 += char  
  
    return sorted(clean1) == sorted(clean2)  
  
# Test cases  
assert is_anagram("listen", "silent") == True  
assert is_anagram("hello", "world") == False
```

```
assert is_anagram("Dormitory", "Dirty Room") == True
```

```
# Edge cases
```

```
assert is_anagram("", "") == True
```

```
assert is_anagram("Hello", "hello") == True
```

```
print("All test cases passed successfully!")
```

output:

```
.venv) PS C:\Users\Dines\OneDrive\Documents\ai assistant coding> & "
python.exe" "c:/Users/Dines/OneDrive/Documents/ai assistant coding/
All test cases passed successfully!
.venv) PS C:\Users\Dines\OneDrive\Documents\ai assistant coding> █
```

Justification:

AI was used to generate test cases for checking anagrams, including normal cases and edge cases. The function processes strings by ignoring case, spaces, and punctuation and was verified using assert tests.

Task Description #4 (Inventory Class – Apply AI to Simulate Real-World Inventory System):

☐ Task: Ask AI to generate at least 3 assert-based tests for an Inventory class with stock management.

☐ Methods:

- o add_item(name, quantity)

- o remove_item(name, quantity)

- o get_stock(name)

Example Assert Test Cases:

```
inv = Inventory()
```

```
inv.add_item("Pen", 10)
```

```
assert inv.get_stock('Pen') == 10
inv.remove_item('Pen', 5)
assert inv.get_stock('Pen') == 5
inv.add_item('Book', 3)
assert inv.get_stock('Book') == 3
```

prompt:

Generate at least 3 assert-based tests for an Inventory class with `add_item()`, `remove_item()`, and `get_stock()` methods. Test adding, removing, and checking item quantities. Then implement the class so all assertions pass.

Input:

```
class Inventory:
```

```
    def __init__(self):
```

```
        self.stock = {}
```

```
    def add_item(self, name, quantity):
```

```
        if name in self.stock:
```

```
            self.stock[name] += quantity
```

```
        else:
```

```
            self.stock[name] = quantity
```

```
    def remove_item(self, name, quantity):
```

```
        if name in self.stock:
```

```
            self.stock[name] -= quantity
```

```
    def get_stock(self, name):
```

```
        return self.stock.get(name, 0)
```

Test cases

```
inv = Inventory()
```

```
inv.add_item("Pen", 10)
```

```
assert inv.get_stock("Pen") == 10
```

```
inv.remove_item("Pen", 5)
```

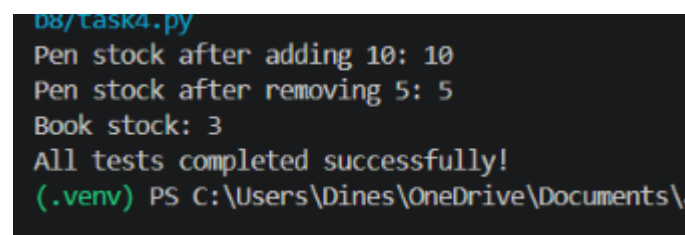
```
assert inv.get_stock("Pen") == 5
```

```
inv.add_item("Book", 3)
```

```
assert inv.get_stock("Book") == 3
```

```
print("All assertions passed successfully!")
```

output:



```
db/task4.py
Pen stock after adding 10: 10
Pen stock after removing 5: 5
Book stock: 3
All tests completed successfully!
(.venv) PS C:\Users\Dines\OneDrive\Documents\
```

Justification:

AI was used to generate test cases and implement an Inventory class. The tests verify adding, removing, and checking stock quantities, ensuring that the inventory system works correctly.

Task Description #5 (Date Validation & Formatting – Apply AI for Data Validation):

☐ Task: Use AI to generate at least 3 assert test cases for `validate_and_format_date(date_str)` to check and convert dates.

☐ Requirements:

- o Validate "MM/DD/YYYY" format.
- o Handle invalid dates.
- o Convert valid dates to "YYYY-MM-DD".

Example Assert Test Cases:

```
assert validate_and_format_date(&quot;10/15/2023&quot;) == &quot;2023-10-15&quot;;
```

```
assert validate_and_format_date(&quot;02/30/2023&quot;) == &quot;Invalid Date&quot;;
```

```
assert validate_and_format_date(&quot;01/01/2024&quot;) == &quot;2024-01-01&quot;;
```

prompt:

Generate at least 3 assert test cases for `validate_and_format_date(date_str)`. Validate the MM/DD/YYYY format, handle invalid dates, and convert valid dates to YYYY-MM-DD. Include edge cases such as leap years.

input:

```
from datetime import datetime
```

```
def validate_and_format_date(date_str):
```

```
    try:
```

```
        date = datetime.strptime(date_str, "%m/%d/%Y")
```

```
        return date.strftime("%Y-%m-%d")
```

```
    except ValueError:
```

```
        return "Invalid Date"
```

```
# Test cases
```

```
assert validate_and_format_date("10/15/2023") == "2023-10-15"
```

```
assert validate_and_format_date("02/30/2023") == "Invalid Date"
```

```
assert validate_and_format_date("01/01/2024") == "2024-01-01"
```

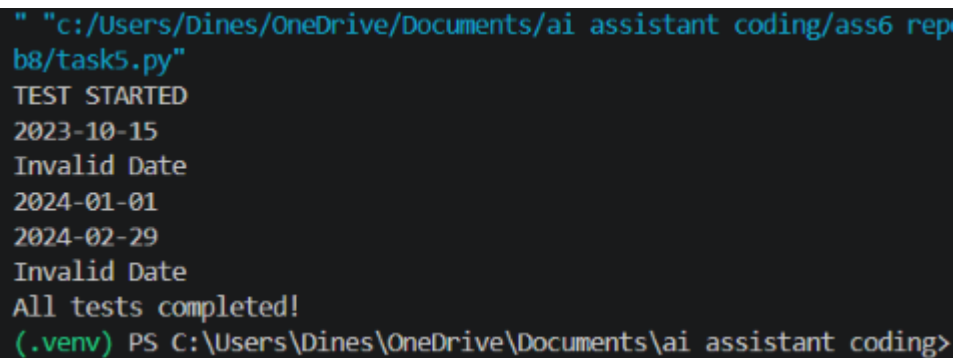
Edge cases

```
assert validate_and_format_date("02/29/2024") == "2024-02-29"
```

```
assert validate_and_format_date("13/01/2023") == "Invalid Date"
```

```
print("All assertions passed successfully!")
```

output:

A terminal window with a dark background and light-colored text. The text shows the execution of a Python script. It starts with a file path, followed by 'TEST STARTED'. Then, it lists several dates: '2023-10-15', 'Invalid Date', '2024-01-01', '2024-02-29', and 'Invalid Date'. After these, it says 'All tests completed!'. The prompt '(.venv) PS C:\Users\Dines\OneDrive\Documents\ai assistant coding>' is visible at the bottom.

```
" "c:/Users/Dines/OneDrive/Documents/ai assistant coding/ass6 rep
b8/task5.py"
TEST STARTED
2023-10-15
Invalid Date
2024-01-01
2024-02-29
Invalid Date
All tests completed!
(.venv) PS C:\Users\Dines\OneDrive\Documents\ai assistant coding>
```

Justification:

AI was used to generate test cases for date validation and formatting. The tests check valid dates, invalid dates, and edge cases such as leap years to ensure reliable date handling.